



Agent Enforcer

Stop the Slop.

Enforce What Your AI Agents Are Allowed to Do — Before They Do It.

Agent Enforcer by Alchemist is an enterprise-grade policy enforcement layer for AI coding agents. It continuously distributes, applies, and locks in your organization's security policies, compliance requirements, and coding standards across every developer and every AI agent on your network — automatically, at the system level, before a single line of code is written.

When your agency deploys AI coding assistants, who's making sure they follow your rules?

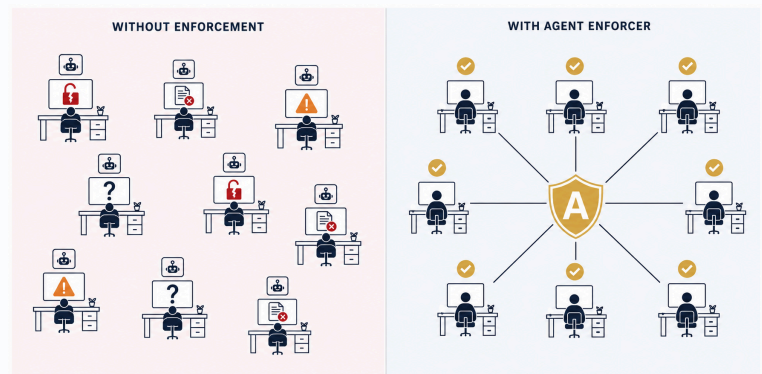
THE PROBLEM

Ungoverned AI Is Already Writing Your Code

Federal agencies are deploying AI coding assistants at scale. Developers are writing more code, faster than ever — powered by large language models that have no inherent knowledge of your security controls, your accreditation requirements, or your internal standards.

Without enforcement:

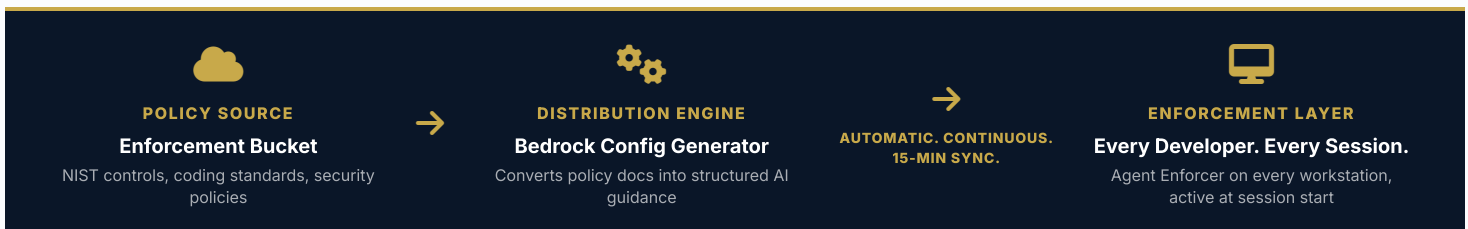
- Every AI session starts with a blank slate. No policies. No guardrails.
- Developers prompt AI differently, producing inconsistent, non-compliant output.
- Security vulnerabilities get written — and shipped — before any human reviews them.
- Audit trails are thin. Accountability is unclear.
- Token costs balloon as agents loop through unguided trial and error.



THE SOLUTION

System-Level Enforcement. Zero Developer Friction.

Agent Enforcer installs as a lightweight system service on developer workstations and CI/CD environments. It connects to your organization's central enforcement bucket, pulls your approved policy configuration, and applies it to every AI coding session — system-wide, without user action.



Policies are defined once by your team or our specialists — then enforced everywhere, automatically.

- ✓ Security requirements baked in before code is written
- ✓ Compliance controls active on every AI session
- ✓ Coding standards applied uniformly across all developers
- ✓ No developer action required — system-level enforcement
- ✓ Updates propagate to every endpoint within 15 minutes
- ✓ Works with Claude Code, Kiro, Cursor, and GitHub Copilot — no rearchitecting

THREE PILLARS OF VALUE

01 — SECURITY & COMPLIANCE ENFORCEMENT

Controls Active Before Code Is Written

AI agents do exactly what they're told. If they aren't told your security requirements, they'll write code that ignores them. Agent Enforcer embeds your controls directly into the AI's operating context:

- **Parameterized queries only** — SQL injection vulnerabilities blocked at the source
- **Secrets via environment variables** — no hardcoded credentials, ever
- **Non-root container execution** — least-privilege Docker enforced by default
- **HTTPS-only endpoints** — enforced protocol standards
- **Approved library lists** — unapproved dependencies flagged before they appear
- **Custom policy documents** — CMMC, FedRAMP, internal STIGs, program-specific controls



Accreditation bodies are beginning to ask: "How do you govern your AI?" Agent Enforcer is your answer.

02 — TOKEN COST REDUCTION

28–35% Savings on AI Token Spend

AI agents without guidance iterate wastefully. They make assumptions, get corrected, retry, loop. Every back-and-forth exchange burns tokens. Agent Enforcer eliminates the most expensive loops before they start by giving the AI the right context from the first prompt.



Conservative example — \$1M/month agency token spend:

SCENARIO	EFFICIENCY GAIN	ANNUAL SAVINGS
Conservative	28%	~\$3.36M/yr
Moderate	30%	~\$3.6M/yr
Optimistic	35%	~\$4.2M/yr

At the conservative estimate, token savings alone exceed the Agency tier platform cost — before accounting for security risk reduction or productivity gains.

03 — UNIFORM CODING STANDARDS

Same Task. Same AI. Consistent, Compliant Output.

Inconsistent code is expensive code. When 50 developers prompt the same AI differently, you get 50 different patterns, 50 different security postures, and exponentially more review burden. Agent Enforcer standardizes output across your entire development organization:

- Mandatory docstrings and documentation standards
- Type hints and annotation requirements
- Named constants — no magic numbers
- Approved architectural patterns per project type
- Style guide compliance before code is committed

Demonstrated result: In controlled testing, enforced AI sessions produced code with zero critical security vulnerabilities versus an average of one critical vulnerability per task in unenforced sessions — running the same AI model, given the same task.

WITHOUT ENFORCER

```
def get_user(uid):
    query = f"SELECT * FROM users WHERE id={uid}"
    result = db.execute(query)
    return result[0]
```

WITH ENFORCER

```
def get_user(user_id: int) -> dict:
    """Retrieve user record by ID."""
    QUERY = "SELECT * FROM users WHERE id = %s"
    result = db.execute(QUERY, (user_id,))
    return result[0]
```

HOW IT WORKS

Four Steps. Compliant by Default.



Define Policy

Your team or Alchemist specialists author enforcement documents in plain English: security requirements, compliance controls, coding standards.



Generate Configuration

Bedrock-powered engine converts policy docs into structured AI guidance — CLAUDE.md, settings, skills, commands.



Distribute Automatically

Agents on every developer machine sync enforcement config every 15 minutes. No developer action required.



Enforce at Session Start

Every AI coding session starts with your policies already active. Compliant by default.

DEPLOYMENT PROFILE

ATTRIBUTE	DETAIL
Platform	Rocky Linux / RHEL 9 (RPM package)
Agent size	< 1 MB installed
Network requirement	Outbound HTTPS to S3 endpoint only
Air-gap compatible	Yes — S3-compatible endpoint configurable
Enforcement latency	Policy updates apply within 15 minutes
User action required	None after initial install
Credentials required	IAM role or instance profile — no user secrets
Audit trail	Sync logs, policy version history in S3
Management console	AWS-native (S3 + CloudWatch)

WHY ALCHEMIST

Alchemist was founded on a single conviction: the value of AI in government is determined not by what the AI *can* do, but by what it's *allowed* to do — and who controls that boundary.

Agent Enforcer puts that control where it belongs: with your security and compliance teams, operating at the system level, before code is written.

We work with agencies to understand their specific requirements — their accreditation frameworks, their internal policies, their existing tooling — and translate that into enforcement documentation that makes every AI coding session compliant by default.

LICENSING & INVESTMENT

Division

Up to 250 seats

\$750,000/year

MOST COMMON

Agency

Up to 1,000 seats

\$2,000,000/year

Enterprise

Up to 5,000 seats

\$4,000,000/year

TIER	SEATS	ANNUAL INVESTMENT
Division	Up to 250	\$750,000
Agency	Up to 1,000	\$2,000,000
Enterprise	Up to 5,000	\$4,000,000
Unlimited	Unlimited	\$7,500,000

Managed Enforcement Specialists (add-on)

An embedded Alchemist specialist who owns your enforcement posture end-to-end — learning your security architecture, compliance frameworks, network topology, and program requirements, then authoring and maintaining enforcement documentation on your behalf. The specialist ensures your AI agents stay current as requirements evolve.

\$1,250,000
/year

Founding Partner Rate — first agency engagement: Contact for details.

© 2026 Alchemist. All rights reserved. Agent Enforcer is a trademark of Alchemist. Results based on controlled testing; individual savings vary by workload composition and enforcement policy depth.